

Rapport de Sprint 1

Federated Learning appliqué à la classification d'images

Cours : 8INF887 – Apprentissage profond

Étudiant : Vinith Naresh Prabakaran

Date : 28 mars 2026

1 Travail réalisé

1.1 Préparation des données

Dans cette étape, le dataset MNIST a été chargé et prétraité. Les images, initialement de dimension 28×28 , sont transformées en tenseurs et normalisées afin d'améliorer la convergence du modèle. Des *DataLoaders* sont utilisés pour organiser les données en mini-batches et optimiser l'entraînement.



```
# On entraîne le modèle par petits groupes de 64 images
BATCH_SIZE = 64
NUM_WORKERS = 0 # pour l'instant

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Device:', device)

# MNIST transforms
transform = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.1307,), (0.3081,))])

# Split entre train et test
train_dataset = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
test_dataset = datasets.MNIST(root='./data', train=False, download=True, transform=transform)

train_loader_full = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, num_workers=NUM_WORKERS)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=NUM_WORKERS)

print('Train samples:', len(train_dataset), 'Test samples:', len(test_dataset))
```

✓ 9.4s

Device: cpu
100.0%
100.0%
100.0%
100.0%
Train samples: 60000 Test samples: 10000

FIGURE 1 – Chargement et préparation des données

1.2 Définition du modèle

Un réseau de neurones MLP a été implémenté. Les images sont d'abord aplaties en vecteurs de taille 784, puis passent à travers deux couches entièrement connectées ($784 \rightarrow 128 \rightarrow 64$) avec des fonctions d'activation ReLU. La couche finale produit une sortie de dimension 10 correspondant aux classes possibles (chiffres de 0 à 9).

```

class SimpleMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten() #transformation vers un vecteur
        self.fc1 = nn.Linear(28 * 28, 128) # 1ere couche
        self.fc2 = nn.Linear(128, 10) # 2eme couche
        self.out = nn.Linear(10, 10) # couche de sortie

    def forward(self, x):
        x = self.flatten(x)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.out(x)
        return x

# nouveau modèle pour chaque client
def create_model():
    return SimpleMLP()

```

0.0s

J'utilise un réseau de neurones de type MLP.

L'image est d'abord transformée en vecteur, puis elle passe à travers plusieurs couches linéaires avec des fonctions d'activation ReLU.

La dernière couche produit une sortie de dimension 10 correspondant aux classes possibles.

FIGURE 2 – Architecture du modèle

1.3 Entraînement centralisé

Un modèle a été entraîné sur l'ensemble des données afin d'obtenir une baseline de référence. L'entraînement est réalisé à l'aide de l'optimiseur SGD et de la fonction de perte CrossEntropy. Après deux époques, le modèle atteint une précision de 97% sur le jeu de test.

Cette baseline permet de comparer les performances avec le Federated Learning.

```

Entraînement centralisé

# nouveau modèle pour chaque client
central_model = create_model().to(device)
optimizer = torch.optim.SGD(central_model.parameters(), lr=0.1)
CENTRAL_EPOCHS = 2

for epoch in range(1, CENTRAL_EPOCHS + 1):
    train_one_epoch(central_model, train_loader_full, optimizer, device)
    loss, acc = evaluate(central_model, test_loader, device)
    print(f'Centralized Epoch {epoch}/{CENTRAL_EPOCHS} -> Test loss: {loss:.4f}, Test acc: {acc:.4f}')

central_loss, central_acc = evaluate(central_model, test_loader, device)
print(
    'Centralized baseline final -> Loss: {:.4f}, Acc: {:.4f}'.format(
        central_loss, central_acc
    )
)

```

4.2s

Centralized Epoch 1/2 -> Test loss: 0.1702, Test acc: 0.9472
 Centralized Epoch 2/2 -> Test loss: 0.0954, Test acc: 0.9700
 Centralized Epoch 2/2 -> Test loss: 0.0954, Test acc: 0.9700
 Centralized baseline final -> Loss: 0.0954, Acc: 0.9700
 Centralized baseline final -> Loss: 0.0954, Acc: 0.9700

FIGURE 3 – Résultats du modèle centralisé

1.4 Simulation des clients

Le dataset est divisé en 5 sous-ensembles afin de simuler un environnement distribué. Chaque client possède une partie des données avec une distribution IID (*Independent and Identically Distri-*

buted), ce qui signifie que les données sont réparties de manière homogène entre les clients.

```
Simulation des clients (IID split)

NUM_CLIENTS = 5
client_subsets = split_dataset_iid(train_dataset, NUM_CLIENTS)
client_loaders = [DataLoader(subset, batch_size=BATCH_SIZE, shuffle=True, num_workers=NUM_WORKERS) for subset in client_subsets]
client_sizes = [len(s) for s in client_subsets]
print('Client data sizes (IID):', client_sizes)

✓ 0.0s

Client data sizes (IID): [12000, 12000, 12000, 12000, 12000]
```

FIGURE 4 – Répartition des données entre les clients

1.5 Entraînement local

Chaque client reçoit une copie du modèle global et l'entraîne localement sur ses propres données pendant une époque. Les paramètres du modèle sont ensuite mis à jour localement sans partager les données.

Cette étape permet d'exploiter les données distribuées tout en préservant leur confidentialité.

```
def local_train_update(global_state_dict, train_loader: DataLoader, device: torch.device, lr: float = 0.1, epochs: int = 1):
    """ fonction de mise à jour de l'entraînement local """
    model = create_model()
    model.load_state_dict(global_state_dict)
    model.to(device)
    optimizer = torch.optim.SGD(model.parameters(), lr=lr)
    for e in range(epochs):
        train_one_epoch(model, train_loader, optimizer, device)
    return model.state_dict(), len(train_loader.dataset)

# test de la mise à jour locale
test_state, test_samples = local_train_update(global_model.state_dict(), client_loaders[0], device, lr=0.1, epochs=1)
print('Local training produced state dict keys:', list(test_state.keys())[:5], 'num samples:', test_samples)

✓ 0.3s

Local training produced state dict keys: ['fc1.weight', 'fc1.bias', 'fc2.weight', 'fc2.bias', 'out.weight'] num samples: 12000

Dans cette étape, chaque client reçoit une copie du modèle global et l'entraîne localement sur ses propres données.
Le modèle est initialisé avec les poids globaux, puis entraîné pendant un nombre limité d'époques.
Une fois l'entraînement terminé, les nouveaux paramètres du modèle ainsi que le nombre d'échantillons utilisés sont retournés au serveur.
Cette étape est essentielle dans le Federated Learning, car elle permet à chaque client d'apprendre à partir de ses données locales sans les partager.
```

FIGURE 5 – Entraînement local des clients

1.6 Agrégation (Federated Averaging)

Les modèles locaux sont combinés à l'aide de l'algorithme *Federated Averaging*. Les poids des modèles sont moyennés de manière pondérée en fonction du nombre d'échantillons de chaque client.

Cette étape permet de construire un modèle global intégrant les connaissances de tous les clients.

```
def federated_average(state_dicts: List[dict], weights: List[float]) -> dict:
    """ fonction d'average des poids des modèles
    """
    # normalise les poids
    total = float(sum(weights))
    if total == 0:
        raise ValueError('Sum of weights must be > 0')
    norm_weights = [w / total for w in weights]

    avg_state = {}
    for key in state_dicts[0].keys():
        # initialiser à zéro du même type que les paramètres originaux
        avg_state[key] = torch.zeros_like(state_dicts[0][key])
        for sd, w in zip(state_dicts, norm_weights):
            avg_state[key] += sd[key].float() * w
    # remettre les types d'origine
    for key in avg_state.keys():
        avg_state[key] = avg_state[key].type(state_dicts[0][key].dtype)
    return avg_state
```

✓ 0.0s

Dans cette étape, les modèles entraînés par chaque client sont combinés pour former un modèle global.

Les paramètres des modèles locaux sont moyennés en tenant compte du nombre de données de chaque client.

Cette méthode, appelée Federated Averaging, permet d'intégrer les connaissances de tous les clients sans partager leurs données.

C'est une étape clé du Federated Learning, car elle permet de construire un modèle global à partir d'apprentissages locaux.

FIGURE 6 – Agrégation des modèles

1.7 Boucle de Federated Learning

Le processus est répété sur plusieurs rounds. À chaque round, le modèle global est envoyé aux clients, entraîné localement, puis les modèles sont agrégés.

Les performances du modèle global sont évaluées à chaque itération, permettant d'observer une amélioration progressive.

```
ROUNDS = 3
FED_LR = 0.1 # valeur d'apprentissage local pour les clients
global_model = create_model()
global_state = global_model.state_dict()

# Suivi de la précision par round
round_results = []
for r in range(1, ROUNDS + 1):
    print(f'--- Federated Round {r}/{ROUNDS} ---')
    local_states = []
    local_sizes = []
    # chaque client s'entraîne localement pendant 1 époque et renvoie ses poids mis à jour
    for i, loader in enumerate(client_loaders):
        sd, n_samples = local_train_update(global_state, loader, device, lr=FED_LR, epochs=1)
        local_states.append(sd)
        local_sizes.append(n_samples)
        print(f' Client {i+1}: trained on {n_samples} samples')
    # Agrégation
    new_global_state = federated_average(local_states, local_sizes)
    global_state = new_global_state
    global_model.load_state_dict(global_state)
    # Évaluation sur le jeu de test
    loss, acc = evaluate(global_model, test_loader, device)
    round_results.append((loss, acc))
    print(f' Round {r} -> Test loss: {loss:.4f}, Test acc: {acc:.4f}')

✓ 6.0s
```

--- Federated Round 1/3 ---

FIGURE 7 – Performances du modèle fédéré

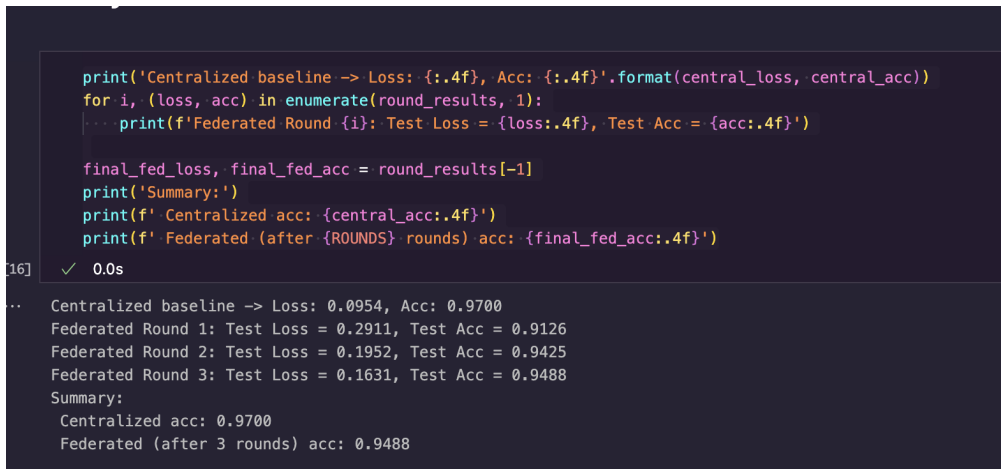
1.8 Analyse des résultats

Le modèle centralisé atteint une précision de 97.0%, tandis que le modèle fédéré atteint 94.88% après 3 rounds.

On observe une amélioration progressive des performances du modèle fédéré, ce qui indique une convergence du modèle global.

Ces résultats sont cohérents avec la théorie du Federated Learning, où l'on observe généralement une performance légèrement inférieure au modèle centralisé en raison de la distribution des données.

Cela illustre le compromis entre performance et confidentialité dans les systèmes distribués.



```
print('Centralized baseline -> Loss: {:.4f}, Acc: {:.4f}'.format(central_loss, central_acc))
for i, (loss, acc) in enumerate(round_results, 1):
    print(f'Federated Round {i}: Test Loss = {loss:.4f}, Test Acc = {acc:.4f}')

final_fed_loss, final_fed_acc = round_results[-1]
print('Summary:')
print(f' Centralized acc: {central_acc:.4f}')
print(f' Federated (after {ROUNDS} rounds) acc: {final_fed_acc:.4f}')

16] ✓ 0.0s

... Centralized baseline -> Loss: 0.0954, Acc: 0.9700
Federated Round 1: Test Loss = 0.2911, Test Acc = 0.9126
Federated Round 2: Test Loss = 0.1952, Test Acc = 0.9425
Federated Round 3: Test Loss = 0.1631, Test Acc = 0.9488
Summary:
Centralized acc: 0.9700
Federated (after 3 rounds) acc: 0.9488
```

FIGURE 8 – Comparaison centralisé vs fédéré

2 Planification du Sprint 2

Le prochain sprint aura pour objectif d'approfondir l'analyse du Federated Learning en se rapprochant de scénarios plus réalistes.

- Implémenter une distribution non-IID des données entre les clients afin de simuler des environnements réels où les données sont hétérogènes
- Comparer les performances du modèle en contexte IID et non-IID
- Étudier l'impact du nombre de clients sur la convergence du modèle global
- Ajouter des visualisations (courbes d'accuracy en fonction des rounds) pour analyser la dynamique d'apprentissage
- Explorer une amélioration du modèle, notamment via l'utilisation d'une architecture plus avancée (par exemple un CNN)

3 Bilan

Ce premier sprint a permis de mettre en place une implémentation complète et fonctionnelle du Federated Learning, incluant l'entraînement local des clients, l'agrégation des modèles via Federated Averaging et l'évaluation des performances.

Les résultats obtenus sont cohérents avec la théorie, avec une convergence progressive du modèle global ainsi qu'un écart limité avec le modèle centralisé.

Ce travail constitue une base solide pour la suite du projet, qui visera à analyser des scénarios plus complexes, notamment avec des données non-IID, afin de mieux refléter les contraintes des environnements réels.